Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures de contrôle itératives et logique d'exécution en programmation

Synthèse exécutive : Ce cours aborde les structures répétitives (boucles) en algorithmique et en langage C. L'objectif est de comprendre pourquoi et comment automatiser la répétition d'instructions sans duplication de code. Le cours détaille le fonctionnement, les cas d'usage stricts et la syntaxe des trois boucles fondamentales : 'while' (tant que), 'do-while' (faire... tant que) et 'for' (pour). Un accent particulier est mis sur la prévention des boucles infinies, la gestion des conditions d'arrêt et l'alignement avec les bonnes pratiques d'évaluation en BUT Informatique.

1. Utilité des structures itératives et problème de la duplication

En programmation, répétition ne doit jamais rimer avec copier-coller. Si un programmeur doit afficher 100 fois un message ou traiter un tableau de 10 000 éléments, dupliquer la ligne de code 10 000 fois rend le programme illisible, lourd et impossible à maintenir. Les structures itératives (ou boucles) permettent d'exécuter un bloc d'instructions plusieurs fois tant qu'une condition logique (expression booléenne) reste vraie. Une boucle mal conçue qui ne modifie jamais sa condition d'arrêt entraîne un blocage complet du programme appelé 'boucle infinie'.

- Principe DRY (Don't Repeat Yourself) : éviter de copier-coller du code
- Condition d'arrêt : expression booléenne qui détermine la sortie de la boucle
- Compteur / Variable de contrôle : variable incrémentée à chaque itération

Cas concret enseignant : *Le professeur a comparé la boucle à un tapis roulant d'usine d'emballage : tant qu'il y a des colis sur le tapis (condition vraie), le bras robotisé attrape un colis (instruction). Si le capteur ne détecte plus de colis ou si le compteur atteint la limite fixée, le robot s'arrête. Sans condition de fin, le robot continuerait à tourner dans le vide et détruirait la machine.*

2. La boucle TANT QUE (while) : Pré-conditionnement

La structure 'while' est utilisée lorsque le nombre exact d'itérations n'est pas connu à l'avance, mais dépend d'un événement dynamique (ex: saisie utilisateur, fin d'un fichier). Sa particularité réside dans le fait que la condition d'entrée est testée d'abord. Si la condition est fausse dès le départ, le bloc d'instructions interne n'est jamais exécuté (0 itération). Il est crucial de s'assurer que le corps de la boucle modifie la valeur testée dans la condition, sinon la boucle devient infinie.

- Pré-condition : La condition est évaluée AVANT chaque exécution du bloc
- Itérations : De 0 à N fois (le bloc peut ne jamais s'exécuter)
- Initialisation : La variable de condition doit être initialisée avant d'entrer dans le while

Cas concret enseignant : *Exemple du contrôle du mot de passe au distributeur de billets : le distributeur demande le code TANT QUE le code saisi est incorrect ET que le nombre d'essais est inférieur à 3. Si l'utilisateur tape le bon code du premier coup, la boucle s'arrête immédiatement.*

3. La boucle FAIRE... TANT QUE (do-while) : Post-conditionnement

Contrairement à la boucle 'while', la boucle 'do-while' exécute toujours le bloc d'instructions au moins une fois avant de tester la condition. Elle est particulièrement adaptée aux menus interactifs et au contrôle de saisie (ex: demander une note comprise entre 0 et 20). En langage C, la syntaxe exige un point-virgule impératif après la parenthèse fermante du 'while' sous peine d'erreur de compilation.

- Post-condition : La condition est évaluée APRÈS l'exécution du bloc

• Itérations : Au moins 1 fois garantie (1 à N fois)
• Point-virgule obligatoire : do { ... } while(condition);

Cas concret enseignant : *Le professeur insiste sur la saisie sécurisée : 'On ne peut pas vérifier si la saisie de l'utilisateur est valide avant qu'il n'ait tapé au moins une valeur au clavier. C'est le cas typique où do-while s'impose naturellement par rapport au simple while.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' regroupe en une seule ligne de commande les trois éléments vitaux d'une itération : l'initialisation de la variable de contrôle, la condition de maintien dans la boucle, et la mise à jour (incrémentation/décrémentation). Elle est privilégiée lorsque le nombre d'itérations est connu avant d'entrer dans la boucle. Les trois parties séparées par des points-virgules dans for(init; cond; inc) sont toutes optionnelles syntaxiquement, mais indispensables en bonne pratique.

• Structure condensée : Initialisation, Condition, Incrémentation sur une seule ligne
• Nombre d'itérations connu : Idéal pour parcourir une plage fixe ou un tableau
• Portée de la variable de boucle (scope) : déclarée généralement dans l'en-tête for(int i = 0; ...)

Cas concret enseignant : *Métaphore du tour de piste d'athlétisme : 'Vous savez à l'avance que vous devez faire exactement 5 tours. Vous préparez le compteur à 1 (init), vous vérifiez si vous avez fait moins de 5 tours (cond), et à chaque ligne d'arrivée franchie, vous ajoutez 1 à votre compteur (inc).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Si la variable 'i' n'est pas modifiée à l'intérieur du corps de la boucle 'while', la condition 'i < N' reste vraie éternellement. C'est une boucle infinie. Le processeur exécute l'instruction en boucle à 100% d'utilisation jusqu'à ce que le programme soit interrompu manuellement.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, exactement. Toute boucle 'for' peut être réécrite sous forme de boucle 'while'. Cependant, pour le confort de lecture et la maintenabilité du code, on utilise 'for' quand le nombre d'itérations est connu d'avance, et 'while' quand la fin dépend d'un événement externe imprevisible.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme en C qui calcule la somme des N premiers entiers positifs (ex: si N=5, calculer 1+2+3+4+5=15). Le programme doit valider que N est strictement supérieur à 0 via une boucle 'do-while', puis calculer la somme à l'aide d'une boucle 'for'.

Résolution pas à pas :
Étape 1 : Déclarer les variables nécessaires (int n, somme = 0, i). Étape 2 : Utiliser do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while(n <= 0); pour garantir un N valide. Étape 3 : Initialiser somme à 0. Étape 4 : Écrire la boucle for(i = 1; i <= n; i++) { somme += i; }. Étape 5 : Afficher le résultat avec printf.

Résultat : Pour N = 5, la boucle 'for' réalise 5 itérations. À i=1 (somme=1), i=2 (somme=3), i=3 (somme=6), i=4 (somme=10), i=5 (somme=15). Le résultat affiché est bien 15. Le code est robuste contre les saisies négatives grâce au do-while.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' teste la condition avant l'exécution du bloc (0 à N itérations), tandis que 'do-while' la teste après l'exécution (1 à N itérations au minimum).

Q : Que se passe-t-il si la condition d'une boucle 'for' est fausse dès le premier test ?

R : Le corps de la boucle n'est jamais exécuté (0 itération) et le programme passe directement à l'instruction située juste après la boucle.

Q : Quel est l'oubli de syntaxe le plus fréquent qui provoque une erreur de compilation sur la boucle 'do-while' en C ?

R : L'oubli du point-virgule obligatoire à la fin de la ligne du while : do { ... } while(condition);